Evaluation Report of EliteFS_09_Mar_AttentionAllYouNeed2-KMIT

Average Score: 7/10 | Attempted: 4 of 4 questions

Question 1

THEORY UNDERSTAND Best Score: 6/10

How is positional encoding done in a transformer?

Desired Answer

Positional encoding is added to the input embeddings to provide information about the position of tokens in the sequence. It uses sine and cosine functions with different frequencies to encode positions, and the resulting position vectors are added element-wise to the input embeddings.

Student's Answer

Positional Encoding is done to ensure the positions of each token in a given text corpus. It is typically done via the sin, cos positional encoding.. where the even positions are given the sin(position/10000^2i/dk) and odd positions are given the cos(pos/1000^2i/dk).. as sin and cos are periodic and differentiable, and provides limited values in a given range.. it provides a good insight on the position of the token provided the entire text.. through Fourier transformation

Evaluations

Feedback 1

★★★☆☆ 6/10

The student is addressing the same concept as the Actual Answer (using sine and cosine functions to encode token positions). They correctly note that even dimensions use sine and odd dimensions use cosine, and they grasp the rationale that sin/cos are periodic and differentiable. However, their formulas are imprecise (they write 10000^2i/dk and even use 1000 for cos instead of 10000), they fail to explicitly state that the positional vectors are added element-wise to the input embeddings, and the mention of a "Fourier transformation" is not part of the standard positional encoding explanation. These errors and omissions limit their answer to an Adequate level of understanding rather than a full explanation.

Feedback 2

★★★☆☆ 6/10

The student is addressing the same concept of positional encoding with sine and cosine functions as in the Actual Answer, so it is relevant. They correctly note that even dimensions use sine and odd dimensions use cosine, and highlight the periodic, differentiable nature of these functions and their role in encoding position. However, they misstate the frequency base (using 1,000 instead of 10,000 in the denominator) and do not mention that these position vectors are added element-wise to the input embeddings. These omissions and the formula inaccuracy indicate only a partial and somewhat imprecise understanding of the full mechanism described in the Actual Answer.

Question 2

THEORY UNDERSTAND **Best Score: 8/10**

Why is there a square root of 'd' in the attention equation?

Desired Answer

The square root of dk(the dimension of the key vector) in the attention equation is used to **normalize the dot product** of the query and key vectors. This prevents the dot products from becoming too large and dominating the softmax function, which could lead to very small gradients and unstable training.

Student's Answer

There is a square root of d in attention equation because when the matrix multiplication of Q and K^T is done, the values become very high.. therefore to normalize the value, we divide it by square root of d, as d is the embedding dimension.

Evaluations

Feedback 1 ★★★★☆ **8/10**

The student is demonstrating understanding of the same concept as the Actual Answer: that the √d term is included to normalize the dot-product of Q and K so that values do not become excessively large. They correctly explain that, without this scaling, the raw QK^T values can grow too high and therefore need to be divided by √d. What they did not explicitly mention—how large dot-products would dominate the softmax and lead to very small gradients and unstable training—is a secondary detail from the Actual Answer. Overall, the student shows a

solid grasp of the core normalization purpose but omits the connection to softmax behavior and gradient stability, so this merits a good (8/10) score.

Feedback 2 ★★★★☆ 8/10

The student is demonstrating understanding of the same concept as the Actual Answer (the normalization of the Q·K dot product by √d to prevent excessively large values). They correctly explain that as the embedding dimension d grows, the raw dot products become large and hence are scaled down by dividing by √d. They have captured the key idea of normalization, though they did not explicitly mention the downstream effect on the softmax (preventing very small gradients and unstable training). This omission is a minor detail, so the answer shows good (but not perfect) understanding.

Question 3

THEORY ANALYZE Best Score: 6/10

What is the difference between feed-forward network and attention network?

Desired Answer

A **feed-forward network (FFN)** is a simple fully connected layer applied to each position independently, without considering other positions in the sequence. In contrast, an **attention network** computes weighted relationships between tokens by considering the entire sequence, allowing each token to focus on relevant parts of the input.

Student's Answer

In feedforward neural network, it has an input layer (512 neurons) called d_model then activation layer relu, then ddf of 2048 neurons, then th output, typically a perceptron is used for feed forward network. Whereas in attention network, each word is mapped with every other word to know how related they are to each other, and provides insights on the same.

Evaluations

Feedback 1 ★★★☆☆ 6/10

The student is addressing the correct concepts—feed-forward networks versus attention networks—as in the Actual Answer, so it is relevant. They correctly note that an FFN is just a sequence of fully connected layers (input → activation → output) applied per token, and that an attention network relates each word to every other word in the sequence. However, they miss the key conceptual contrast that a feed-forward layer processes each position independently (ignoring other tokens), whereas attention explicitly computes weighted relationships across all positions. Instead, they focus on specific layer sizes and activations for the FFN, which, while not wrong, do not convey the conceptual point about independence. Their description of attention correctly captures mapping words to one another but omits mention of weighting the relationships. Overall, they show a basic understanding of both architectures but do not fully explain the independence versus sequence-wide weighting distinction presented in the Actual Answer.

Feedback 2

★★½☆☆ 5/10

The student is addressing the same concepts as the Actual Answer (feed-forward vs. attention networks) but only partly captures the core ideas. They correctly state that an attention network maps each word to every other word to gauge relevance, matching the idea of computing weighted relationships over the whole sequence. However, their description of the feed-forward network focuses solely on layer sizes and activation (512→ReLU→2048→output), without explaining the key fact that FFNs operate on each position independently and do not consider other positions. Because they miss this essential conceptual point for FFNs, their answer shows only an adequate (partial) understanding of the Actual Answer's main distinction.

Question 4

THEORY UNDERSTAND Best Score: 9/10

Describe the structure of the feed-forward network?

Desired Answer

The feed-forward network in a Transformer consists of two linear layers with a ReLU activation in between. The input is passed through the first linear layer, then through a ReLU activation, and finally through the second linear layer. The FFN operates independently on each position in the sequence.

Student's Answer

The structure of feedforward layer is : 1st linear layer of 512 neurons known as d_model then ReLu Activation layer then hidden layer of 2048 neurons then again 512 neurons, then output.. FFn=ReLu(Xinput * W1)W2 where X input is the input, W1 is the weight with respect to 1st layer.. and W2 is the weight wrt 2nd layer.. Total weights are=>2*(512 *2048)

Evaluations

Feedback 1

7/10

The student is addressing the same concepts as the Actual Answer (a two-layer feed-forward network with a ReLU activation in between). They correctly state that the input is multiplied by a first weight matrix W1, passes through ReLU, then is multiplied by a second weight matrix W2. They even identify the common Transformer dimensions (512 → 2048 → 512) and sketch the parameter count. However, they omit mention of the fact that the FFN operates independently at each sequence position, and their description of 'hidden layer of 2048 then again 512 then output' is a bit confused in wording. Overall, they capture the core structure (two linear layers with a ReLU) accurately but miss one of the key details and have minor clarity issues, placing their answer at a good but not comprehensive level of understanding.

Feedback 2

9/10

The student is addressing exactly the same feed-forward network structure as the Actual Answer (two linear layers with a ReLU in between). They correctly describe the first linear transform, the ReLU activation, and the second linear transform, even giving the typical Transformer dimensions (512→2048→512) and a formula FFn=ReLU(X·W1)·W2. They go slightly beyond by computing the total weight count. The one minor omission is the note that the FFN is applied independently at each sequence position, but that is a secondary detail. Overall, the student shows clear and mostly complete understanding of the mechanism described in the Actual Answer.